

Memoria Práctica 1

Sistemas Informáticos II

Sistema de Pagos Visa — Django REST Framework

Alumno: Alejandro Pardo, Pablo Pollo

Asignatura: Sistemas Informáticos II

Universidad: Universidad Autónoma de Madrid (UAM)

Curso: 2025-2026

Fecha: 19 de febrero de 2026

Arquitectura del sistema:

VM1 (puerto 12022): Base de datos PostgreSQL (puerto 15432)

VM2 (puerto 22022): Aplicación Django + Backend REST (puerto 28000)

VM3 (puerto 32022): Frontend REST (puerto 38000)

Ejercicio 1: Despliegue Local (0.5 pts)

Se ha desplegado el proyecto P1-base en el ordenador host conectándolo a la base de datos PostgreSQL en VM1 (localhost:15432). Se ha verificado el correcto funcionamiento de todos los endpoints de la aplicación.

Endpoints probados

http://localhost:8080/visaApp/tarjeta/

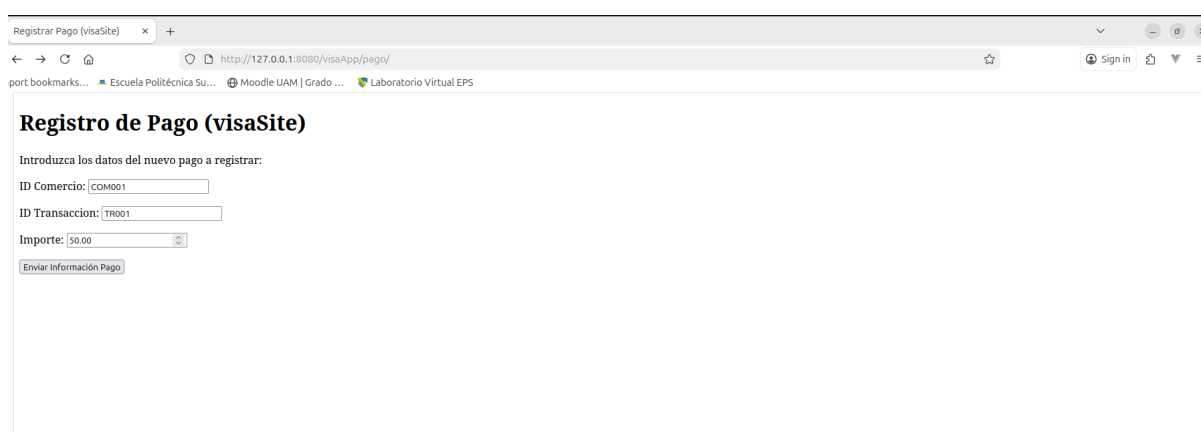
Formulario de tarjeta. Se introduce una tarjeta válida (número: 3974 8228 3584 7557, nombre: Gabriel Vallejo Cozar, fecha: 07/29, código: 861). Tras el envío, redirige al formulario de pago.



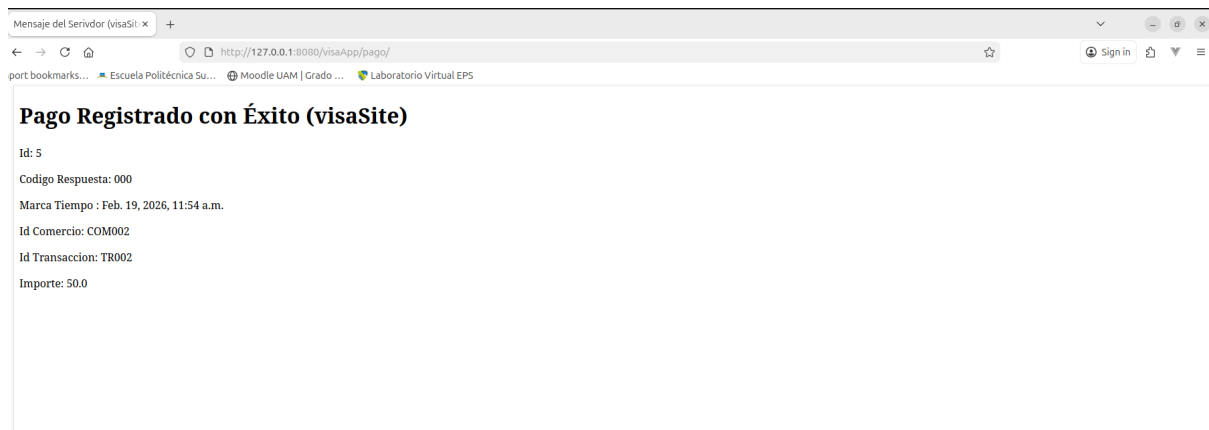
A screenshot of a web browser showing the 'Aportar Información Tarjeta' form. The browser's address bar displays 'http://127.0.0.1:8080/visaApp/tarjeta/'. The form title is 'Introduzca la Información de la Tarjeta de la Persona que Paga (visaSite)'. It contains four input fields: 'Número de Tarjeta:' with the value '3974 8228 3584 7557', 'Nombre:' with 'Gabriel Vallejo Cozar', 'Fecha de Caducidad:' with '07/29', and 'Código de Autorización:' with '861'. Below these fields is a button labeled 'Enviar Información Tarjeta'.

http://localhost:8080/visaApp/pago/

Formulario de pago. Se rellenan los datos (idComercio, idTransaccion, importe) y se registra correctamente.

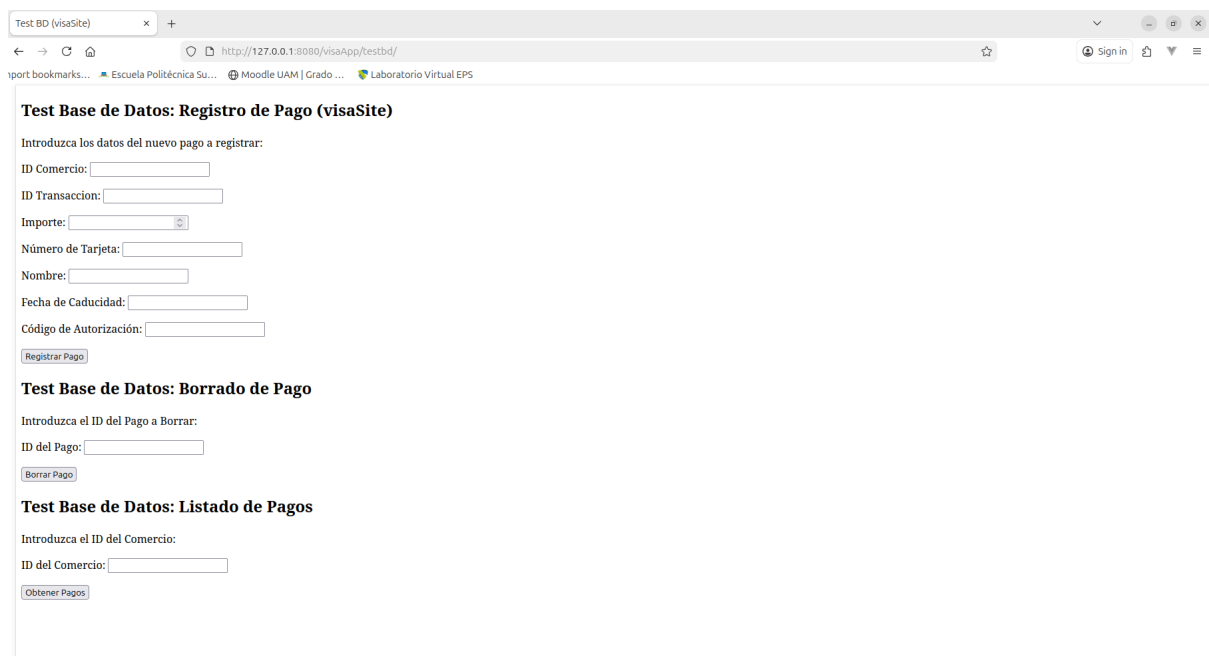


A screenshot of a web browser showing the 'Registrar Pago (visaSite)' form. The browser's address bar displays 'http://127.0.0.1:8080/visaApp/pago/'. The form title is 'Registro de Pago (visaSite)'. It includes the instruction 'Introduzca los datos del nuevo pago a registrar:'. There are three input fields: 'ID Comercio:' with the value 'COM001', 'ID Transaccion:' with 'TR001', and 'Importe:' with '50.00'. A button labeled 'Enviar Información Pago' is located at the bottom of the form.



http://localhost:8080/visaApp/testbd/

Formulario combinado que permite verificar tarjeta y registrar pago en un solo paso.



Contenido del fichero env

```
DATABASE_SERVER_URL='postgres://alumnodb:alumnodb@localhost:15432/si2db'
DEBUG=True
SECRET_KEY='django-insecure-alczftn)j1#5v%xmK@5j(n*px43c8kxgi_ua4%khc+t7g_)s9d'
```

```
(venv) e462971@4-15-4-15:~/Descargas/si2$ ssh si2@localhost -p 22022 "cat /home/si2/repo/p1base/si2_alumnos-main/P1-base/env"
(si2@localhost) Password:
DATABASE_SERVER_URL='postgres://alumnodb:alumnodb@10.0.2.2:15432/si2db'
DEBUG=True
SECRET_KEY='django-insecure-alczftn)j1#5v%xmK@5j(n*px43c8kxgi_ua4%khc+t7g_)s9d'
```

Ejercicio 2: Tests Locales (0.5 pts)

Se han ejecutado los tests proporcionados con el proyecto P1-base desde el host, con la base de datos en VM1. Todos los tests (18) pasan correctamente.

Comando ejecutado

```
python manage.py test
```

Resultado: 18 tests ejecutados — OK

```
● ^C(venv) e462971@4-15-4-15:~/Descargas/si2_alumnos-main/P1-basepython manage.py test
Found 18 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
...Error: Registrando pago: null value in column "tarjeta_id" of relation "pago" vi
olates not-null constraint
DETAIL: Failing row contains (5, C001, T001, 123, 2026-02-19 10:57:34.287319+00, 00
0, null).

.....
-----
Ran 18 tests in 0.245s

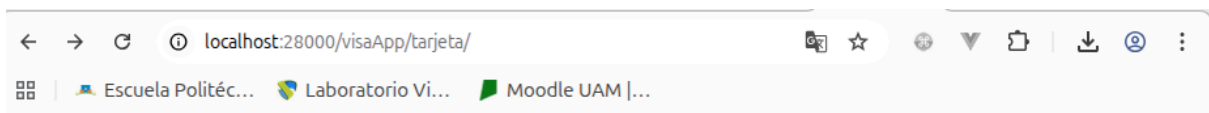
OK
Destroying test database for alias 'default'...
○ (venv) e462971@4-15-4-15:~/Descargas/si2_alumnos-main/P1-base$
```

Ejercicio 3: Despliegue en Máquinas Virtuales (0.5 pts)

Se ha desplegado el proyecto P1-base en VM2 usando Gunicorn como servidor WSGI, con la base de datos en VM1. El despliegue se realizó mediante un repositorio Git bare en VM2 con un hook post-receive que automatiza migraciones, collectstatic y reinicio de Gunicorn.

Proceso de despliegue

1. Se creó un repositorio bare en VM2: /home/si2/repo/p1base.git
2. Se configuró un hook post-receive para automatizar el despliegue
3. Se configuró Gunicorn como servicio systemd en VM2
4. Se realizó git push desde el host a VM2
5. Se verificó el funcionamiento en <http://localhost:28000/visaApp/tarjeta/>



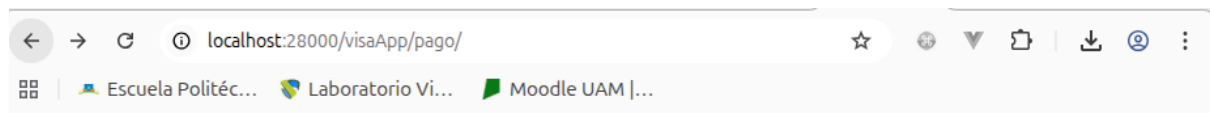
Introduzca la Información de la Tarjeta de la Persona que Paga (visaSite)

Número de Tarjeta:

Nombre:

Fecha de Caducidad:

Código de Autorización:



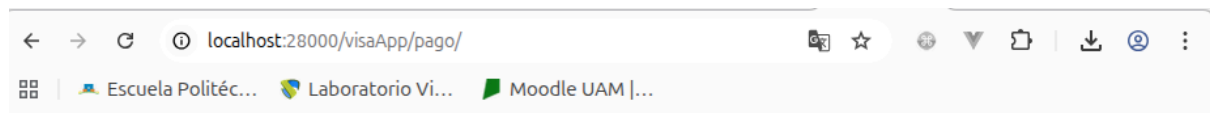
Registro de Pago (visaSite)

Introduzca los datos del nuevo pago a registrar:

ID Comercio:

ID Transaccion:

Importe:



Pago Registrado con Éxito (visaSite)

Id: 6

Codigo Respuesta: 000

Marca Tiempo : Feb. 19, 2026, 1:46 p.m.

Id Comercio: COM001

Id Transaccion: TR001

Importe: 50.0

Contenido del fichero env en VM2

```
DATABASE_SERVER_URL='postgres://alumnodb:alumnodb@10.0.2.2:15432/si2db'  
DEBUG=True  
SECRET_KEY='django-insecure-alczftnjl1#sv%xmke5j(n*px43c8kxgi_ua4%khc+t7g_)s9d'
```

Ejercicio 4: Benchmark de Base de Datos (0.5 pts)

Se ha medido el tiempo de lectura de 1000 entradas de la tabla tarjeta en diferentes escenarios, ejecutando cada medida 7 veces.

4.1: SQL Directo contra VM1

Script en Python usando psycopg2 para conectar directamente a PostgreSQL en VM1 (localhost:15432) y realizar 1000 consultas individuales.

Ejecución	Tiempo (s)
1	0.602075
2	0.269466
3	0.278212
4	0.255507
5	0.258259
6	0.518052
7	0.243202
Media	0.3464
Desv. estándar	0.1484

Fichero env:

DATABASE_SERVER_URL='postgres://alumnodb:alumnodb@localhost:15432/si2db'

```
e462971@4-15-4-15:~/Descargas/si2/si2_alumnos-main/P1-base$ source env/bin/activate
for i in {1..7}; do echo "Ejecución $i:"; python read_1000_entries.py; done
Ejecución 1:
Tiempo: 0.602075 segundos
Ejecución 2:
Tiempo: 0.269466 segundos
Ejecución 3:
Tiempo: 0.278212 segundos
Ejecución 4:
Tiempo: 0.255507 segundos
Ejecución 5:
Tiempo: 0.258259 segundos
Ejecución 6:
Tiempo: 0.518052 segundos
Ejecución 7:
Tiempo: 0.243202 segundos
```

4.2: SQL Directo contra Neon (nube)

No fue posible ejecutar este escenario desde los PCs del laboratorio debido a que nos salía que las conexiones salientes al puerto 5432 estaban bloqueadas por el firewall de la red de la universidad. La conexión a Neon (PostgreSQL en la nube) requiere acceso TCP al puerto 5432 de servidores externos, lo cual no está permitido en la red del laboratorio.

Intentamos la conexión a ep-wild-wildflower-ag37rvuz-pooler.c-2.eu-central-1.aws.neon.tech pero no supimos cómo continuar en este apartado porque obtuvimos el error "Connection timed out".

Tras probarlo de nuevo el mismo día de la entrega en otro ordenador de otro laboratorio si pudimos ejecutarlo y tras cambiar el fichero env correspondiente y la db_config del propio script obtuvimos lo siguiente:

Fichero env:

```
# IMPORTANT: this file should not be in a repository
# To remove a file named env from a Git repository
# but keep it in the source (local system), follow these steps:
# Remove the file from Git tracking but keep it locally
## git rm --cached env
# Add 'env' to .gitignore (so it's not tracked again)
## echo "env" >> .gitignore
# Commit the changes
## git commit -m "Removed env from Git tracking and added to .gitignore"
# Push the changes to the remote repository
## git push
# use sqlite 3
## DATABASE_SERVER_URL=sqlite:///db.sqlite3
# use postgres
DATABASE_SERVER_URL='postgresql://neondb_owner:npg_oBSXz8L06PKH@ep-wild-wildflower-ag37rvuz-pooler.c-2.eu-central-1.aws.neon.tech/si2db?sslmode=require&'
# The client does not need to store data in any database
# so let us define a sqlite in order to avoid warning messages
DEBUG=True
SECRET_KEY = 'django-insecure-alczftnjl1$v%xmK@5j(n*px43c8kxgi_ua4%khc+t7g_)s9d'
```

db_config modificado para Neon:

```
import psycopg2
import time

db_config = {
    'dbname': 'si2db',
    'user': 'neondb_owner',
    'password': 'npg_oBSXz8L06PKH',
    'host': 'ep-wild-wildflower-ag37rvuz-pooler.c-2.eu-central-1.aws.neon.tech',
    'port': 5432,
    'sslmode': 'require'
}

try:
    conn = psycopg2.connect(**db_config)
    cursor = conn.cursor()

    # Leer las primeras 1000 entradas
    cursor.execute("SELECT * FROM tarjeta LIMIT 1000")
    rows = cursor.fetchall()

    search_query = 'SELECT * FROM tarjeta WHERE "numero" = %s'

    # Medir tiempo
    start_time = time.time()

    for row in rows:
        id_value = row[0]
        cursor.execute(search_query, (id_value,))
        cursor.fetchone()

    end_time = time.time()

    print(f"Tiempo: {end_time - start_time:.6f} segundos")
```

```
• (venv) e462971@1-19-1-19:~/Descargas/P1_SI2_AlejandroPablo/SI2-P1-entrega/P1-base$ for i in {1..7}; do echo "Ejecución $i:"; python read_1000_entries_from_db.py; done
Ejecución 1:
Tiempo: 33.931734 segundos
Ejecución 2:
Tiempo: 37.951515 segundos
Ejecución 3:
Tiempo: 34.056428 segundos
Ejecución 4:
Tiempo: 34.029601 segundos
Ejecución 5:
Tiempo: 38.547446 segundos
Ejecución 6:
Tiempo: 33.994088 segundos
Ejecución 7:
Tiempo: 38.076395 segundos
```

4.3: ORM Django contra VM1

Script en Python usando el ORM de Django (Tarjeta.objects.get) para realizar 1000 consultas individuales contra PostgreSQL en VM1.

Ejecución	Tiempo (s)
1	0.739616

2	0.710675
3	0.860234
4	0.998259
5	0.751483
6	0.727401
7	0.699945
Media	0.7839
Desv. estándar	0.1083

Fichero env:

DATABASE_SERVER_URL='postgres://alumnodb:alumnodb@localhost:15432/si2db'

```
(venv) e462971@4-15-4-15:~/Descargas/si2/si2_alumnos-main/P1-base$ for i in {1..7}; do echo "Ejecución $i:"; python read_1000_orm.py; done
Ejecución 1:
Tiempo: 0.739616 segundos
Ejecución 2:
Tiempo: 0.710675 segundos
Ejecución 3:
Tiempo: 0.860234 segundos
Ejecución 4:
Tiempo: 0.998259 segundos
Ejecución 5:
Tiempo: 0.751483 segundos
Ejecución 6:
Tiempo: 0.727401 segundos
Ejecución 7:
Tiempo: 0.699945 segundos
(venv) e462971@4-15-4-15:~/Descargas/si2/si2_alumnos-main/P1-base$
```

Script ORM reescrito (4.3)

```
import os, sys, time, django
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'visaSite.settings')
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
django.setup()
from visaApp.models import Tarjeta
tarjetas = list(Tarjeta.objects.all()[:1000])
numeros = [t.numero for t in tarjetas]
start_time = time.time()
for numero in numeros:
    tarjeta = Tarjeta.objects.get(pk=numero)
end_time = time.time()
print(f'Tiempo: {end_time - start_time:.6f} segundos')
```

Comparativa y comentario

Escenario	Media (s)	Desv. est.	Ratio
SQL Directo	0.3464	0.1484	1.00x
ORM Django	0.7839	0.1083	2.26x

El ORM de Django introduce una sobrecarga significativa respecto al SQL directo. La media del escenario ORM (0.7839s) es aproximadamente 2.3 veces mayor que la del SQL directo (0.3464s). Esta sobrecarga se debe a varias capas de abstracción que Django añade: la instanciación de objetos Python por cada fila recuperada, la validación de campos, la

traducción de la consulta ORM a SQL, y la gestión del caché de querysets. Aunque el ORM es más lento, ofrece ventajas en productividad del desarrollador, seguridad (prevención de inyección SQL), portabilidad entre bases de datos y mantenibilidad del código.

Cuestión 1: ¿Son suficientes 7 repeticiones?

Para evaluar si 7 repeticiones proporcionan una estimación fiable, analizamos la variabilidad de las medidas y calculamos los intervalos de confianza al 95% usando la distribución t de Student ($t=2.447$ para $n=7$, $gl=6$).

Intervalos de confianza al 95%

Escenario	Media (s)	IC 95%	Coef. var.
SQL Directo	0.3464	[0.2092, 0.4836]	42.8%
ORM Django	0.7839	[0.6838, 0.8841]	13.8%

Análisis del Escenario SQL Directo: El coeficiente de variación es 42.8%, lo que indica una variabilidad relativamente alta. Esto se debe principalmente a la primera ejecución (0.602s), que es significativamente más lenta que las siguientes (efecto de 'cold start' o calentamiento de caché). Si se eliminara este outlier, la variabilidad sería mucho menor.

Análisis del Escenario ORM: El coeficiente de variación es 13.8%, indicando una variabilidad menor y medidas más consistentes. El intervalo de confianza es más estrecho, lo que sugiere mayor estabilidad.

Conclusión: Con 7 repeticiones se obtiene una estimación razonablemente fiable para el escenario ORM, cuyo coeficiente de variación es bajo. Para el escenario SQL directo, la primera medida introduce un valor atípico debido al calentamiento de caché, lo que amplía el intervalo de confianza.

Una mejora sería descartar la primera medida (warm-up) y/o aumentar a 10-15 repeticiones para reducir el efecto de valores atípicos. No obstante, incluso con 7 medidas, la diferencia entre ambos escenarios es clara y estadísticamente significativa: el ORM es consistentemente más lento que SQL directo.

Ejercicio 5: Tests en Máquinas Virtuales (0.5 pts)

Se han ejecutado los tests del proyecto P1-base desde VM2, con la base de datos en VM1. Todos los tests (18) pasan correctamente.

Comando ejecutado en VM2

```
ssh si2@localhost -p 22022
cd /home/si2/repo/p1base/si2_alumnos-main/P1-base
python manage.py test
```

Resultado: 18 tests ejecutados — OK

```
(venv) e46297104-15-4-15:~/Descargas/si2$ ssh si2@localhost -p 22022
(si2@localhost) Password:
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.8.0-85-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of jue 19 feb 2026 13:48:35 CET

System load:          0.0
Usage of /:           34.6% of 8.65GB
Memory usage:        22%
Swap usage:          0%
Processes:           122
Users logged in:      1
IPv4 address for enp0s3: 10.0.2.15
IPv6 address for enp0s3: fd17:625c:f037:2:a00:27ff:fe5a:c794

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

https://ubuntu.com/engage/secure-kubernetes-at-the-edge

El mantenimiento de seguridad expandido para Applications está desactivado

Se pueden aplicar 189 actualizaciones de forma inmediata.
106 de estas son actualizaciones de seguridad estándares.
Para ver estas actualizaciones adicionales, ejecute: apt list --upgradable

Active ESM Apps para recibir futuras actualizaciones de seguridad adicionales.
Vea https://ubuntu.com/esm o ejecute «sudo pro status»

Last login: Thu Feb 19 13:42:35 2026 from 10.0.2.2
si2@si2-ubuntu-vm-2:~$ cd /home/si2/repo/p1base/si2_alumnos-main/P1-base
python manage.py test
Found 18 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
...Error: Registrando pago: null value in column "tarjeta_id" of relation "pago" violates not-null constraint
DETAIL:  Failing row contains (5, C001, T001, 123, 2026-02-19 12:49:21.499676+00, 000, null).

*****
Ran 18 tests in 0.256s

OK
Destroying test database for alias 'default'...
```

Ejercicio 6: Tests WS-Backend (obligatorio)

Se han ejecutado los tests proporcionados con el proyecto P1-ws-backend (aplicación visaAppWSBackend) en VM2, con la base de datos en VM1. Todos los tests (6) pasan correctamente.

Comando ejecutado

```
cd /home/si2/repo/p1base/si2_alumnos-main/P1-ws-backend
python manage.py test visaAppWSBackend.test_views
```

Resultado: 6 tests ejecutados — OK

```
* Documentation: https://help.ubuntu.com
* Management:   https://landscape.canonical.com
* Support:       https://ubuntu.com/pro

System information as of jue 19 feb 2026 14:15:45 CET

System load:          0.0
Usage of /:            34.6% of 8.65GB
Memory usage:         23%
Swap usage:           0%
Processes:            122
Users logged in:      1
IPv4 address for enp0s3: 10.0.2.15
IPv6 address for enp0s3: fd17:625c:f037:2:a00:27ff:fe5a:c794

* Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
  just raised the bar for easy, resilient and secure K8s cluster deployment.

  https://ubuntu.com/engage/secure-kubernetes-at-the-edge

El mantenimiento de seguridad expandido para Applications está desactivado

Se pueden aplicar 189 actualizaciones de forma inmediata.
106 de estas son actualizaciones de seguridad estándares.
Para ver estas actualizaciones adicionales, ejecute: apt list --upgradable

Active ESM Apps para recibir futuras actualizaciones de seguridad adicionales.
Vea https://ubuntu.com/esm o ejecute «sudo pro status»

Last login: Thu Feb 19 14:01:03 2026 from 10.0.2.2
si2@si2-ubuntu-vm-2:~$ cd /home/si2/repo/p1base/si2_alumnos-main/P1-ws-backend
python manage.py test visaAppWSBackend.test_views
Found 6 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
.....
-----
Ran 6 tests in 0.276s

OK
Destroying test database for alias 'default'...
si2@si2-ubuntu-vm-2:~/repo/p1base/si2_alumnos-main/P1-ws-backend$ exit
logout
Connection to localhost closed.
```

Ejercicio 7: Endpoints REST (obligatorio)

Se han probado todos los endpoints del API REST del backend desplegado en VM2 (<http://localhost:28000/restapiserver/>). Se muestran las pruebas realizadas con curl desde el host.

1. POST /restapiserver/tarjeta/ — Tarjeta válida

URL:	<code>http://localhost:28000/restapiserver/tarjeta/</code>
Método:	POST
Datos:	<code>{"numero":"3974 8228 3584 7557","nombre":"Gabriel Vallejo Cozar","fechaCaducidad":"07/29","codigoAutorizacion":"861"}</code>
Resultado:	<code>{"message":"Datos encontrados en la base de datos"} - HTTP 200</code>

2. POST /restapiserver/tarjeta/ — Tarjeta inválida

URL:	<code>http://localhost:28000/restapiserver/tarjeta/</code>
Método:	POST
Datos:	<code>{"numero":"0000 0000 0000 0000","nombre":"Falso","fechaCaducidad":"01/99","codigoAutorizacion":"000"}</code>
Resultado:	<code>{"message":"Datos no encontrados en la base de datos"} - HTTP 404</code>

3. POST /restapiserver/pago/ — Crear pago

URL:	<code>http://localhost:28000/restapiserver/pago/</code>
Método:	POST
Datos:	<code>{"idComercio":"COM002","idTransaccion":"TR002","importe":75.0,"tarjeta_id":"3974 8228 3584 7557"}</code>
Resultado:	<code>{"id":8,"idComercio":"COM002",...} - HTTP 200</code>

4. GET /restapiserver/comercio/COM001

URL:	<code>http://localhost:28000/restapiserver/comercio/COM001</code>
Método:	GET
Datos:	N/A
Resultado:	<code>[{"id":6,"idComercio":"COM001",...}] - HTTP 200</code>

5. DELETE /restapiserver/pago/8

URL:	<code>http://localhost:28000/restapiserver/pago/8</code>
Método:	DELETE
Datos:	N/A
Resultado:	<code>{"message":"Pago eliminado correctamente."} - HTTP 200</code>

6. DELETE /restapiserver/pago/99999

URL:	<code>http://localhost:28000/restapiserver/pago/99999</code>
Método:	DELETE

Datos:	N/A
Resultado:	{"message": "Pago no encontrado."} — HTTP 404

```
(venv) e462971@4-15-4-15:~/Descargas/si2$ curl -X POST http://localhost:28000/restapiserver/tarjeta/ \
-H "Content-Type: application/json" \
-d '{"numero": "3974 8228 3584 7557", "nombre": "Gabriel Vallejo Cozar", "fechaCaducidad": "07/29", "codigoAutorizacion": "861"}'
{"message": "Datos encontrados en la base de datos"}(venv) e462971@4-15-4-15:~/Descargas/si2$ curl -X POST
(venv) e462971@4-15-4-15:~/Descargas/si2$ curl -X POST http://localhost:28000/restapiserver/tarjeta/ \
-H "Content-Type: application/json" \
-d '{"numero": "0000 0000 0000 0000", "nombre": "Falso", "fechaCaducidad": "01/99", "codigoAutorizacion": "000"}'
{"message": "Datos no encontrados en la base de datos"}(venv) e462971@4-15-4-15:~/Descargas/si2$ curl -X P
(venv) e462971@4-15-4-15:~/Descargas/si2$ curl -X POST http://localhost:28000/restapiserver/pago/ \
-H "Content-Type: application/json" \
-d '{"idComercio": "COM001", "idTransaccion": "TR001", "importe": 50.0, "tarjeta_id": "3974 8228 3584 7557"}'
{"message": "Error al registrar pago."}(venv) e462971@4-15-4-15:~/Descargas/si2$ curl http://localhost:280
00/restapiserver/comercio/COM001 curl http://localhost:28000/restapiserver/comercio/COM001
[{"id":6,"idComercio":"COM001","idTransaccion":"TR001","importe":50.0,"marcaTiempo":"2026-02-19T13:46:44.433233+01:00","codigoRespuesta":"000","tarjeta":"3974 8228 3584 7557"}](venv) e462971@4-15-4-15:~/Descarg
as/si2$ curl -X DELETE http://localhost:28000/restapiserver/pago/1 curl -X DELETE http://localhost:28000/restapiserver/pago/1
{"message": "Pago no encontrado."}(venv) e462971@4-15-4-15:~/Descargas/si2$ curl -X DELETE http://localhos
t:28000/restapiserver/pago/99999 curl -X DELETE http://localhost:28000/restapiserver/pago/99999
{"message": "Pago no encontrado."}(venv) e462971@4-15-4-15:~/Descargas/si2$ curl -X POST http://localhost:
(venv) e462971@4-15-4-15:~/Descargas/si2$ curl -X POST http://localhost:28000/restapiserver/pago/ \
-H "Content-Type: application/json" \
-d '{"idComercio": "COM002", "idTransaccion": "TR002", "importe": 75.0, "tarjeta_id": "3974 8228 3584 7557"}'
{"id":8,"idComercio":"COM002","idTransaccion":"TR002","importe":75.0,"tarjeta":"3974 8228 3584 7557","cod
igoRespuesta":"000"}(venv) e462971@4-15-4-15:~/Descargas/si2$ curl -X DELETE http://localhost:28000/resta
(venv) e462971@4-15-4-15:~/Descargas/si2$ curl -X DELETE http://localhost:28000/restapiserver/pago/8
```

Ejercicio 8: Tests WS-Frontend (1.0 pts)

Se han ejecutado los tests proporcionados con el proyecto P1-ws-frontend (aplicación visaAppWSFrontend). Los tests se ejecutan con la base de datos en VM1 y el backend REST en VM2 (localhost:28000). Todos los tests pasan correctamente.

Comando ejecutado

```
cd P1-ws-frontend  
python manage.py test visaAppWSFrontend.test_views
```

Resultado: 8 tests ejecutados — OK

```
e462971@4-15-4-15:~/Descargas/si2/si2_alumnos-main/P1-ws-frontend$ cd /home/alumnos/  
e462971/Descargas/si2/si2_alumnos-main/P1-ws-frontend  
source venv/bin/activate  
python manage.py test visaAppWSFrontend.test_views  
Found 8 test(s).  
Creating test database for alias 'default'...  
System check identified no issues (0 silenced).  
.....  
-----  
Ran 8 tests in 0.441s  
  
OK  
Destroying test database for alias 'default'...
```

Ejercicio 9: Sistema Completo WS (1.0 pts)

Se ha desplegado el sistema completo con la arquitectura: Navegador → Frontend (localhost:8081) → Backend REST (VM2:28000) → BD (VM1:15432). Se han probado todos los endpoints del frontend verificando que la comunicación con el backend REST funciona correctamente.

Endpoints del frontend probados

http://localhost:8081/visaAppWSFrontend/tarjeta/

Formulario de tarjeta — redirige al formulario de pago tras verificar tarjeta vía REST

http://localhost:8081/visaAppWSFrontend/pago/

Formulario de pago — registra el pago mediante llamada POST al backend REST

http://localhost:8081/visaAppWSFrontend/testbd/

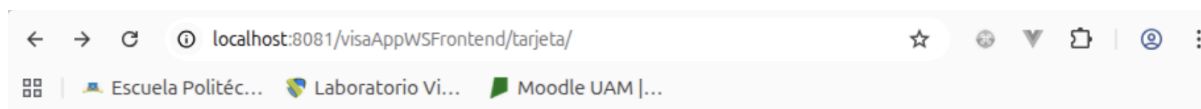
Formulario combinado — tarjeta + pago en un solo paso

http://localhost:8081/visaAppWSFrontend/testbd/getpagos/

Listar pagos de un comercio — consulta vía GET al backend REST

http://localhost:8081/visaAppWSFrontend/testbd/delpago/

Eliminar pago — solicitud DELETE al backend REST



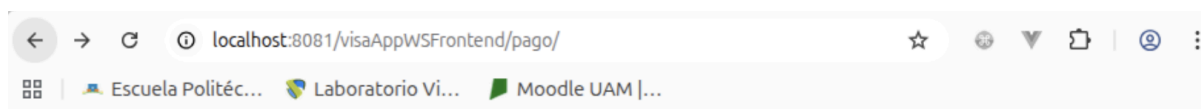
Introduzca la Información de la Tarjeta de la Persona que Paga (visaSite-Frontend)

Número de Tarjeta:

Nombre:

Fecha de Caducidad:

Código de Autorización:



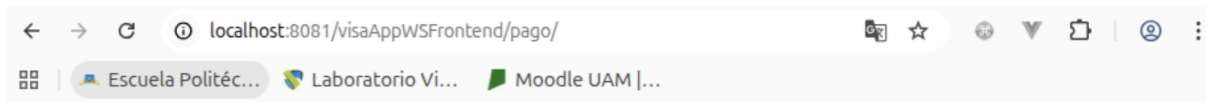
Registro de Pago (visaSite-Frontend)

Introduzca los datos del nuevo pago a registrar:

ID Comercio:

ID Transaccion:

Importe:



Pago Registrado con Éxito (visaSite-Frontend)

Id: 9

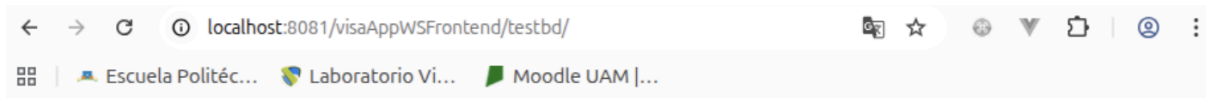
Codigo Respuesta: 000

Marca Tiempo :

Id Comercio: COM100

Id Transaccion: TR001

Importe: 99.99



Test Base de Datos: Registro de Pago (visaSite-Frontend)

Introduzca los datos del nuevo pago a registrar:

ID Comercio:

ID Transaccion:

Importe:

Número de Tarjeta:

Nombre:

Fecha de Caducidad:

Código de Autorización:

Test Base de Datos: Borrado de Pago

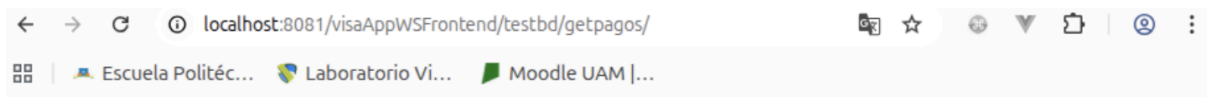
Introduzca el ID del Pago a Borrar:

ID del Pago:

Test Base de Datos: Listado de Pagos

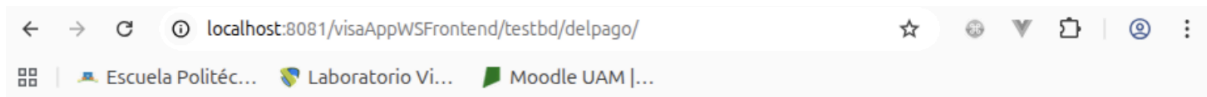
Introduzca el ID del Comercio:

ID del Comercio:



Pagos Registrados (visaSite-Frontend)

id	IdComercio	IdTransaccion	Importe	Marca Tiempo	Codigo Respuesta
27	COM200	TR200	120.5	2026-02-19T14:46:41.861386+01:00	000



¡Pago eliminado correctamente!

Contenido del fichero env del Frontend

```

DATABASE_SERVER_URL='postgres://alumnodb:alumnodb@localhost:15432/si2db'
RESTAPIBASEURL='http://localhost:28000/restapiserver/'
DEBUG=True
SECRET_KEY='django-insecure-alczftn)j1#$v%xmk@5j(n*px43c8kxgi_ua4%khc+t7g_)s9d'

```

Contenido del fichero env del Backend (VM2)

```

DATABASE_SERVER_URL='postgres://alumnodb:alumnodb@10.0.2.2:15432/si2db'
DEBUG=True
SECRET_KEY='django-insecure-alczftn)j1#$v%xmk@5j(n*px43c8kxgi_ua4%khc+t7g_)s9d'

```